SAE S6 :

Phase 2 : Google Gemini

Nous avons reformulé les consignes et nous avons donné le même prompt (au début, car ensuite chacun a eu des demandes différentes) à toutes les IA pour avoir une application en python, car c’est le langage le plus utilisé et facilement testable.

Prompt original :

“

Développe une application Python complète pour générer et visualiser des diagrammes de Voronoï.

SPÉCIFICATIONS FONCTIONNELLES :
- Lire un fichier texte contenant des points au format : une paire de coordonnées par ligne (ex: "2,4" ou "5.3,4.5")
- Calculer le diagramme de Voronoï à partir de ces points
- Afficher une visualisation interactive du diagramme
- Permettre l'export en format SVG et image (PNG/JPG)
- Fournir une interface utilisateur conviviale

EXIGENCES TECHNIQUES :
- Code Python respectant les meilleures pratiques (PEP 8, type hints, docstrings)
- Architecture modulaire et maintenable
- Suite de tests complète (tests unitaires et d'intégration)
- Gestion des erreurs et cas limites
- Documentation du code

LIVRABLES ATTENDUS :
- Code source complet et structuré
- Fichier requirements.txt avec les dépendances
- Tests automatisés
- Instructions d'installation et d'utilisation

“

Début des tests :

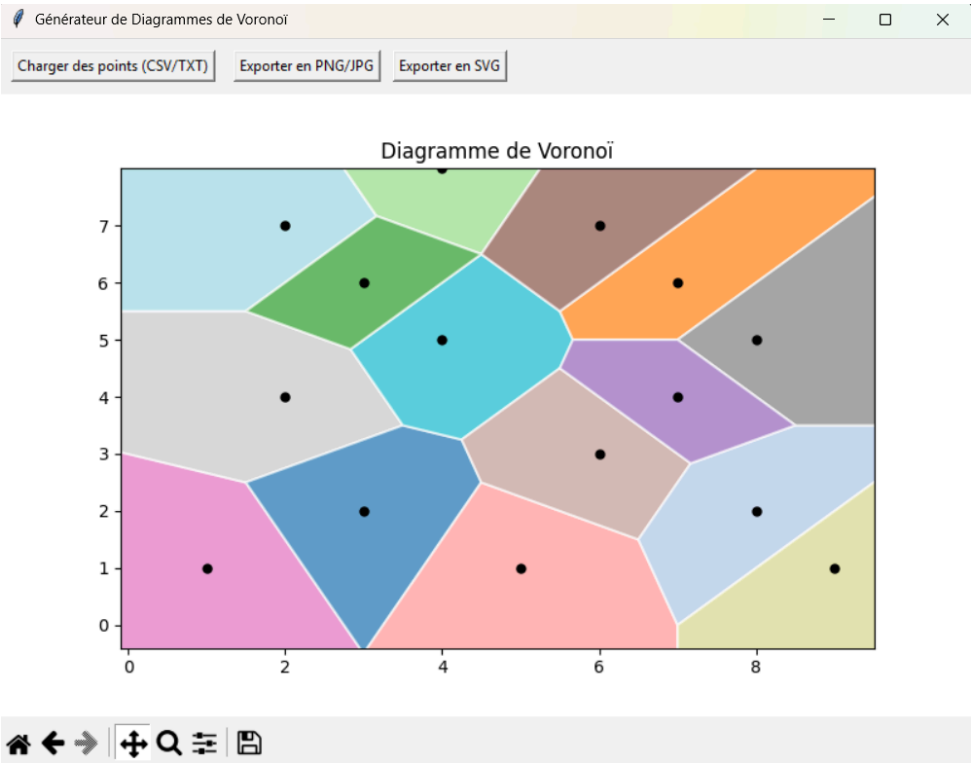
=== TEST IA : Gemini ===

Date et heure de début : 24/02/2026 - 14h57

Date et heure de début : 24/02/2026 - 15h13

Phase initial :

- Temps de génération des codes avec le premier prompt : 50 secondes
- Temps pour implémenter (sur Intellij) et tester : 8 min 40 -> certains packages l'IA ne la pas dit d'installer donc nous avons dû l'installer (ou plutôt Intelleji a suggéré de le faire comme Matplotlib ou Scipy)
- Temps de correction : 30 secondes -> La première version du rendu était un diagramme sans couleur, donc Gemini devait alors fournir un code avec ces modifications.
- Est ce que le code fonctionne sans bug dès le début ? -> Oui le code a fonctionné dès le début du premier coup et les tests fournis par Gemini ont fonctionné également.
- **Fonctionnalités clés** :
    - **Gestion des données**
        - Importation des fichiers : L'application peut lire des fichiers de texte (ou CSV) qui contient les coordonnées de points (x,y par ligne)
        - Exportation des fichiers : L'application permet d'exporter le diagramme sous format PNG, JPG …
    - **Calcul**
        - Algorithme : L'application génère un diagramme de Voronoï avec le fichier fournis qui contient les coordonnées de points
    - **Partie visuelle**
        - Affichage coloré : L'application remplit automatiquement chaque cellule avec une couleur (après correction)
        - Repères : Un point noir est présent au centre de chaque cellule
        - Outils de navigation : Une barre d'outil est présente pour zoomer, se déplacer dans le diagramme, et également un outil qui permet de modifier l'interface du diagramme (le recadrer)



(Capture d'écran pour voir les fonctionnalités - partie visuelle)

- Comme nous l'avons demandé, Gemini a également fourni des tests et tout fonctionnement correctement et respecte parfaitement le diagramme.

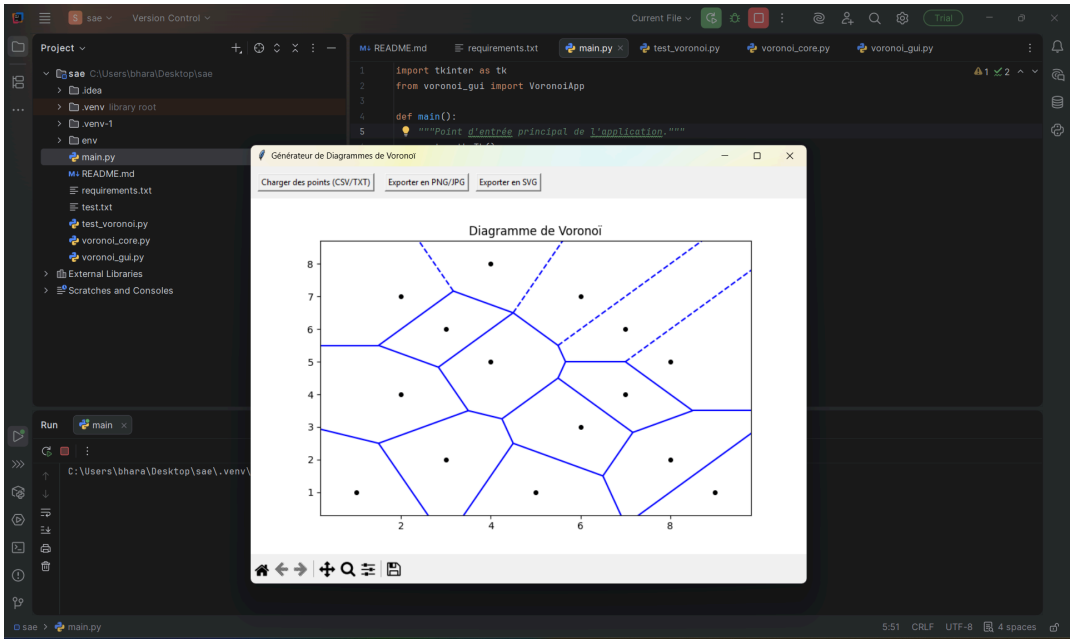
```
✓ 5 tests passed   5 tests total, 6 ms
---------------------------- test session starts ----------------------------
collecting ... collected 5 items

test_voronoi.py::test_load_points_valid PASSED                          [ 20%]
test_voronoi.py::test_load_points_invalid PASSED                        [ 40%]
test_voronoi.py::test_compute_voronoi_success PASSED                    [ 60%]
test_voronoi.py::test_compute_voronoi_insufficient_points PASSED        [ 80%]
test_voronoi.py::test_compute_voronoi_collinear PASSED                  [100%]
```

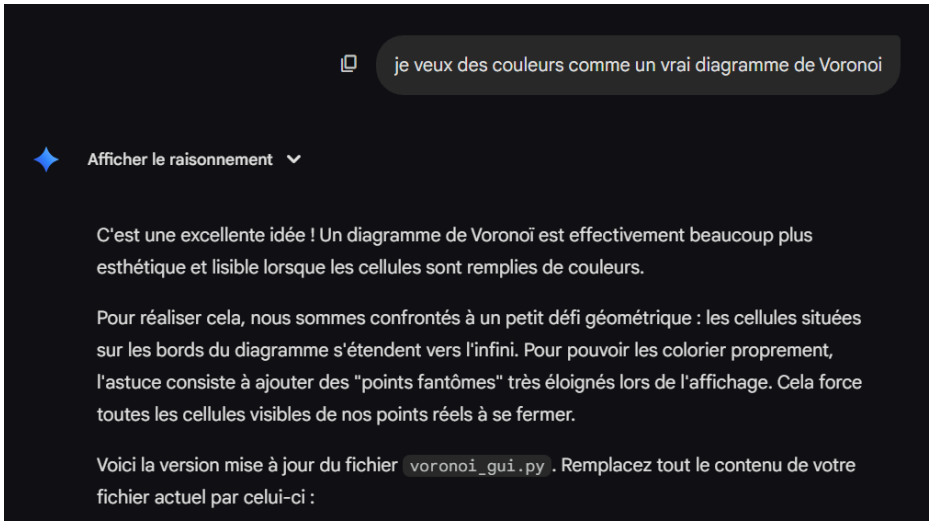
(Capture d'écran des tests)

Phase de correction :

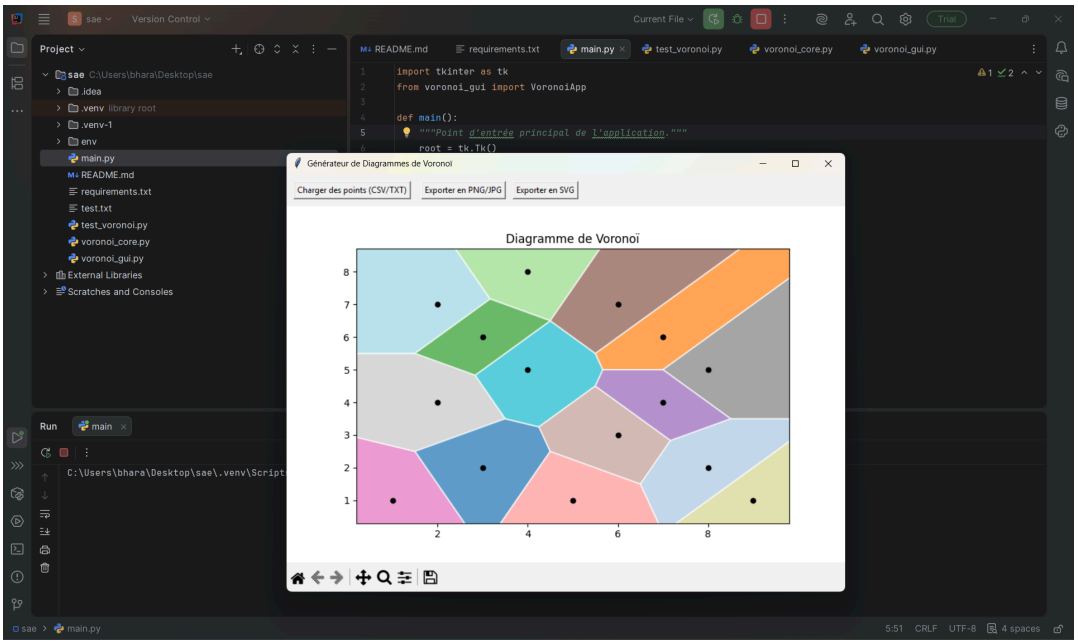
- La seule correction à faire c'était de rendre le diagramme (avant l'export coloré), car sa première version n'était pas coloré. Pour cela nous devions faire un prompt pour que Gemini nous passe un code modifié pour les couleurs.



(Capture d'écran de la première version du rendu)



(Capture d'écran du prompt demandé pour le changement)



(Capture d'écran de la version finale après le changement)

Comparaison IA vs Humain :

- **Vitesse de code**
    - IA (Gemini) : Gemini était capable de nous fournir une application complète (avec tous les fichiers séparés, les tests et les explications)
    - Humain : Même les meilleurs développeurs auraient mis plusieurs heures à développer l'architecture complète de cette application sans bug, (la réflexion, la logique, les tests, configuration des packages comme Matplotlib)
- **Connaissance**
    - IA (Gemini) : Gemini connaît les bibliothèques qu'il faut utiliser et sait également comment les lier. Par exemple, l'IA a su utiliser scipy pour relier Matplotlib à Tkinter.
    - Humain : Un développeur devra lire la documentation technique, consulter des forums, pour déjà trouver comment l'algorithme devra fonctionner, ensuite comprendre que faire etc …
- **Rôle important**
    - IA (Gemini) : Ici l'IA code suite aux demandes de l'humain, elle exécute seulement les ordres de l'humain, donc sans guide, sans bon prompt il n'y aurait pas eu de codes de qualité.
    - Humain : Même si l'humain paraît inférieur par rapport aux aspects vus précédemment, il reste le maître d'action. En effet, lors du premier essai, nous avons constaté que l'application finale nous fournissait un diagramme sans couleurs, c'est l'humain qui a trouvé l'erreur pas Gemini. C'est l'humain qui lui a dit qu'il y avait une erreur en demandant de changer cela. Donc l'humain est la personne qui va valider à la fin le projet en question (ici notre application).

Donc nous voyons bien que l'IA ne remplace pas le développeur, ou plutôt il lui facilite la tâche, mais l'humain derrière est devenu le "superviseur" de l'IA et doit savoir gérer le code, concevoir l'idée et guider l'IA avec les prompts (clairs et précis). Même s'il ya un gain de temps énorme, sans l'humain derrière qui guide une IA, il n'y aurait rien.